# PRNN Programming Assignment 1

PRNN 2026

Prathosh AP, ECE, IISc

March 2026

## General Instructions

- **Library Restrictions:** You are strictly forbidden from using `scikit-learn` or any equivalent high-level machine learning libraries. You must implement the core algorithms from scratch using `numpy` and `scipy`. Standard plotting libraries (`matplotlib`, `seaborn`) are permitted.
- **Deliverables:** For each question, you must submit the specified plots, the precise numerical values requested, and the mathematical or programmatic explanations for the behaviors you observe. All of this must be included in a 4-page report and submitted along with the code as separate files.

## 1 Phase 1: Bio-Optic Sensor Calibration (Dataset 1)

A hospital has procured a new batch of low-cost, non-invasive optical biosensors designed to measure peripheral blood oxygenation. However, the raw voltage output from the sensor needs to be calibrated against a highly accurate (but expensive and invasive) arterial blood gas test. While the theoretical relationship between the raw voltage and the true blood oxygen level is linear, these low-cost sensors are inherently noisy due to ambient light interference. This dataset serves as your pipeline's foundational sanity check.

**1.1 Gradient Descent vs. Closed Form:** Implement Ordinary Least Squares (OLS) using both the closed-form normal equations and vectorized batch gradient descent. Train both on Dataset 1. Report the $L_2$ norm of the difference between the final weight vectors produced by both methods.

**1.2 Primal to Dual Translation:** Convert Dataset 1 into a binary classification problem by thresholding the target variable at its median. Formulate the hard-margin SVM dual optimization problem. Use `scipy.optimize.minimize` to solve for the Lagrange multipliers ($\boldsymbol{\mu}$). Extract and print the exact indices of the support vectors from the training set.

## 2 Phase 2: Oncology Array (Dataset 2)

For a specialized oncology study, the clinic ran high-throughput genomic sequencing on a cohort of patients to predict a continuous tumor severity score. Because the sequencing is incredibly expensive, the sample size is severely constrained ($N = 500$). However, the microarray outputs $D = 1000$ distinct protein expression levels. This represents a pathological optimization scenario. Not only is the data matrix "fat" ($D > N$), but biological pathways dictate that many of these proteins are co-expressed, introducing extreme multicollinearity.

**2.3 The Singularity Trap:** Attempt to compute the closed-form OLS solution on Dataset 2. Document the specific Python exception that halts execution and explain the mathematical property of the dataset causing it. Next, implement $L_2$ regression to rescue the inversion. Plot the condition number of the matrix $(X^T X + \lambda I)$ as a function of $\lambda$ evaluated over a logarithmic grid from $10^{-5}$ to $10^2$.

**2.4 Lasso Sparsity:** Implement $L_1$ regression using Coordinate Descent. Run it on Dataset 2. Generate a regularization path plot (Weight Values vs. $\log(\lambda)$). Programmatically identify and report the exact value of $\lambda$ at which exactly 50% of the feature weights are driven to absolute zero.

## 3 Phase 3: High-Frequency ICU Telemetry (Dataset 3)

The Intensive Care Unit (ICU) requires a real-time triage model. Patients are continuously monitored across 50 high-frequency physiological metrics. You must classify the patient's state into one of three categories: *Stable* (Class 0), *At-Risk* (Class 1), or *Critical* (Class 2). The core difficulty lies in the "Stable" class, which consists of two vastly different physiological states: patients who are awake and resting, and patients who are in a state of chemically induced deep sleep.

**3.5 Vectorized Logistic Regression:** Filter Dataset 3 to include only Class 1 and Class 2. Implement binary Logistic Regression with $L_2$ regularization using gradient descent. Plot the learning curve. Demonstrate empirically how the curve behaves when you increase the learning rate to a point that violates the Lipschitz smoothness condition.

**3.6 GMM Initialization & The E-Step:** Implement the EM algorithm for a Gaussian Mixture Model. Initialize a GMM with $K = 4$ on Dataset 3 using random means and spherical covariances. Execute *exactly one* E-step and *one* M-step. Report the initial marginal log-likelihood of the dataset, and the exact log-likelihood after this single iteration.

**3.7 EM Convergence & Crashes:** Run your GMM on Dataset 3 for $K = 2, 4, 6$. Plot the log-likelihood over iterations to empirically verify monotonic convergence. Programmatically force one of the covariance matrices to zero variance mid-training, and document the exact Python error that breaks the Multivariate Normal PDF computation.

**3.8 Decision Boundaries:** Extract the first two dimensions of Dataset 3. Train a GMM ($K = 3$) on this subset. Plot a dense grid of the resulting Bayes Optimal decision boundaries, and overlay the contour lines of the learned Gaussian covariance matrices on top of the data scatter plot.

**3.9 SVM with Slack KKT Verification:** Filter Dataset 3 for Class 1 and Class 2. Implement the soft-margin SVM dual and train it with $C = 1.0$. Programmatically evaluate your optimized $\boldsymbol{\mu}$ array to verify the KKT conditions. Print the index and $\mu$ value of: (a)

one point strictly inside the margin, (b) one point exactly on the margin boundary, and (c) one point safely classified outside the margin.

**3.10** **Mercer's Condition:** Implement the RBF (Gaussian) Kernel function. Compute the full $N \times N$ Kernel matrix $K$ for a 1,000-sample subset of Dataset 3. Compute the eigenvalues of $K$. Show programmatically that all eigenvalues are non-negative, verifying that your kernel matrix is Positive Semi-Definite.

**3.11** **Hyperparameter Topography:** Write a script to perform a grid search over $C \in \{0.1, 1, 10, 100\}$ and $\sigma \in \{0.001, 0.01, 0.1, 1, 10\}$ for your RBF SVM. Generate a 2D heatmap plot of the validation accuracy across this grid. Identify the precise hyperparameter coordinate $(C, \sigma)$ where the model exhibits the most severe overfitting.

## 4 Phase 4: Unscaled Electronic Health Records (Dataset 4)

The hospital administration wants to predict which incoming emergency room patients are at risk for severe sepsis complications. You are given a massive dump of EHR data containing 64 variables. This presents the messy reality of clinical data: heavy class imbalance (90% routine, 10% severe) and wildly unscaled features. Column 3 might be "Age" (0–100), while Column 12 is "White Blood Cell Count" (4000–11000).

**4.12** **Multiclass Extension & Scaling:** Extend your logistic regression to multiclass using the One-vs-Rest strategy. Run it on the raw Dataset 4. Then, implement a `StandardScaler` from scratch, apply it to Dataset 4, and run the model again. Report the exact number of iterations required to converge in both scenarios.

**4.13** **Vectorization Benchmarking:** Implement a K-Nearest Neighbors (KNN) classifier. Write two versions: one that computes pairwise Euclidean distances using nested `for` loops, and one that uses purely vectorized matrix operations. Time both implementations on a 2,000-sample test split of Dataset 4 and report the speedup factor.

**4.14** **The Curse of Scale:** Using your vectorized KNN ($K = 5$), evaluate the model on both the raw and standardized versions of Dataset 4. Report the exact difference in validation accuracy.

**4.15** **Bayes Optimal & Underflow:** Implement a Naive Bayes classifier with Gaussian Class-conditional densities computing log-probabilities to prevent underflow. Temporarily alter your code to multiply the raw probabilities instead. Report the exact feature dimension $D$ at which the joint probability evaluating Dataset 4 underflows exactly to 0.0 in Python.

## 5 Phase 5: Bias-Variance Decomposition

The hospital administration requires a rigorous mathematical risk assessment of your predictive models before they can be deployed in the ICU. They need to understand the fundamental sources of your model's prediction errors: is the model too rigid to capture complex biological realities (Bias), is it dangerously sensitive to the specific subset of patients in the training cohort (Variance), or is the error arising from inherent, unmeasurable physiological noise (Irreducible Error)?

**5.16** **Empirical Bias-Variance via Bootstrapping (Dataset 1):** You will empirically estimate the bias-variance tradeoff. Implement a polynomial feature extraction function. Train a simple Linear Regression model ($d = 1$) and a complex unregularized Polynomial Regression model ($d = 15$) on Dataset 1. Instead of a single train/test split, generate

$B = 100$ bootstrap samples of your training data. Train 100 separate models for both $d = 1$ and $d = 15$. Programmatically compute and report the empirical squared Bias and empirical Variance of the predictions on the hold-out test set for both polynomial degrees.

**5.17** **Frequentist vs. Bayesian Uncertainty (Dataset 1):** In the frequentist framework, your parameter estimate $\hat{\theta}$ is a random variable dependent on the data sample. Calculate the frequentist variance of your estimated slope parameter from the $B = 100$ bootstrapped $d = 1$ models above. Next, implement a Bayesian MAP estimator for the same slope parameter using a Gaussian prior. Calculate the analytical Posterior Variance of the parameter. Plot both distributions and document the mathematical difference in what "expectation" means in these two paradigms.